PROJETO TMBA - Google Play Store App

Projeto desenvolvido por:

- Débora Jesus, nº2250064
- João Vasques, nº2250055
- Rui Filipe Parada, nº2211025

LLM: DistilBERT

1. Imports and Data Upload

```
# Instalação de bibliotecas adicionais
!pip install -q transformers datasets scikit-learn
!pip install datasets
!pip install torchvision
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.8.5)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.29.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.0.2)
Requirement already satisfied: pyarrow>=21.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (24.0.0)
Requirement already satisfied: dill<0.4.2,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: httpx<1.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.28.1)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.7.0)
Requirement already satisfied: multiprocess<0.70.20 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2026.2.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2026.2.0,>=2023.1.0->datasets) (
Requirement already satisfied: huggingface-hub<2.0,>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (1.16.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (26.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2026.2.0,>=2023.1.0->datasets) (3
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets) (4.13.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets) (2026.5.20)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets) (3.15)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1.0.0->datasets) (0.16.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.25.0->datasets) (1.5.0)
Requirement already satisfied: typer>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.25.0->datasets) (0.25.1)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.25.0->datasets) (4.15.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.4.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2.5.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2026.2)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023.1.
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=202
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=202
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->datasets) (1.17.0)
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer>=0.20.0->huggingface-hub<2.0,>=0.25.0->datasets) (8.4.0
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.20.0->huggingface-hub<2.0,>=0.25.0->datasets)
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.20.0->huggingface-hub<2.0,>=0.25.0->datasets) (13.9.
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer>=0.20.0->huggingface-hub<2.0,>=0.25.0->datasets
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer>=0.20.0->huggingface-hub<2.0,>=0
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer>=0.20.0->huggingface-hub<2.0,>
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=13.8.0->typer>=0.20.0->huggingface
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.27.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: torch==2.12.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.12.0)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch==2.12.0->torchvision) (3.29.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch==2.12.0->torchvision) (4.15.0)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch==2.12.0->torchvision) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch==2.12.0->torchvision) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch==2.12.0->torchvision) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch==2.12.0->torchvision) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch==2.12.0->torchvision) (2025.3.0)
Requirement already satisfied: cuda-toolkit==13.0.2 in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cudart,cufft,cufile,cupti,curand,cusolve
Requirement already satisfied: nvidia-cublas<=13.1.1.3,>=13.1.0.3 in /usr/local/lib/python3.12/dist-packages (from torch==2.12.0->torchvision) (13.1.1.3)
```

```
# Imports principais
from google.colab import drive
from google.colab import files
import pandas as pd
import numpy as np
import io

# Pré-processamento e métricas
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    cohen_kappa_score,
    confusion_matrix
)

# Hugging Face - tokenização, modelo e treino
```

```
from transformers import (
    DistilBertTokenizerFast,
    DistilBertForSequenceClassification,
    AutoTokenizer,
    TrainingArguments,
    Trainer
)

# Conversão para dataset compatível com Hugging Face
from datasets import Dataset
import sys
import torch

# Outros imports
import shap
import scipy as sp
```

```
# Upload do dataset (em csv)

uploaded = files.upload()

review_df = pd.read_csv(io.BytesIO(uploaded['google_play_store_apps.csv']), sep=',')
```

Escolher Ficheiros google_play_store_apps.csv

google_play_store_apps.csv(text/csv) - 6436021 bytes, last modified: 16/03/2026 - 100% done

```
Saving google_play_store_apps.csv to google_play_store_apps.csv
```

```
review_df.head()
```

	App	Translated_Review	Sentiment	Sentiment_Polarity	Sentiment_Subjectivity
0	10 Best Foods for You	I like eat delicious food. That's I'm cooking ...	Positive	1.00	0.533333
1	10 Best Foods for You	This help eating healthy exercise regular basis	Positive	0.25	0.288462
2	10 Best Foods for You	Works great especially going grocery store	Positive	0.40	0.875000
3	10 Best Foods for You	Best idea us	Positive	1.00	0.300000
4	10 Best Foods for You	Best way	Positive	1.00	0.300000

Passos seguintes: Gerar código com `review_df` | New interactive sheet

2. Pré-processamento

```
# Remover reviews com sentimento neutro
review_df = review_df[review_df['Sentiment'] != 'Neutral']

# Visualizar as primeiras linhas
display(review_df.head(10))
```

	App	Translated_Review	Sentiment	Sentiment_Polarity	Sentiment_Subjectivity
0	10 Best Foods for You	I like eat delicious food. That's I'm cooking ...	Positive	1.000000	0.533333
1	10 Best Foods for You	This help eating healthy exercise regular basis	Positive	0.250000	0.288462
2	10 Best Foods for You	Works great especially going grocery store	Positive	0.400000	0.875000
3	10 Best Foods for You	Best idea us	Positive	1.000000	0.300000
4	10 Best Foods for You	Best way	Positive	1.000000	0.300000
5	10 Best Foods for You	Amazing	Positive	0.600000	0.900000
8	10 Best Foods for You	good you.	Positive	0.700000	0.600000
9	10 Best Foods for You	Useful information The amount spelling errors ...	Positive	0.200000	0.100000
10	10 Best Foods for You	Thank you! Great app!! Add arthritis, eyes, im...	Positive	0.750000	0.875000
11	10 Best Foods for You	Greatest ever Completely awesome maintain heal...	Positive	0.992188	0.866667

3. Fine Tuning

3.1. Codificação dos Rótulos

```
# Codificar sentimentos em valores numéricos
le = LabelEncoder()
review_df["label"] = le.fit_transform(review_df["Sentiment"])

# Mostrar primeiras linhas
display(review_df.head(10))
```

	App	Translated_Review	Sentiment	Sentiment_Polarity	Sentiment_Subjectivity	label
0	10 Best Foods for You	I like eat delicious food. That's I'm cooking ...	Positive	1.000000	0.533333	1
1	10 Best Foods for You	This help eating healthy exercise regular basis	Positive	0.250000	0.288462	1
2	10 Best Foods for You	Works great especially going grocery store	Positive	0.400000	0.875000	1
3	10 Best Foods for You	Best idea us	Positive	1.000000	0.300000	1
4	10 Best Foods for You	Best way	Positive	1.000000	0.300000	1
5	10 Best Foods for You	Amazing	Positive	0.600000	0.900000	1
8	10 Best Foods for You	good you.	Positive	0.700000	0.600000	1
9	10 Best Foods for You	Useful information The amount spelling errors ...	Positive	0.200000	0.100000	1
10	10 Best Foods for You	Thank you! Great app!! Add arthritis, eyes, im...	Positive	0.750000	0.875000	1
11	10 Best Foods for You	Greatest ever Completely awesome maintain heal...	Positive	0.992188	0.866667	1

3.2. Separação dos Dados em Treino e Teste

```
# Separar dados de treino (80%) e teste (20%)
train_df = review_df.sample(frac=0.8, random_state=42)
test_df = review_df.drop(train_df.index)
```

```
# Mostrar dimensões dos conjuntos
print("train_df:", train_df.shape)
print("→", train_df.shape[0], "linhas (documentos de treino) e", train_df.shape[1], "colunas (atributos seleccionadas)")

print("\ntest_df:", test_df.shape)
print("→", test_df.shape[0], "linhas (documentos de teste) e", test_df.shape[1], "colunas (atributos seleccionadas)")
```

```
train_df: (25815, 6)
→ 25815 linhas (documentos de treino) e 6 colunas (atributos seleccionadas)

test_df: (6454, 6)
→ 6454 linhas (documentos de teste) e 6 colunas (atributos seleccionadas)
```

3.3. Conversão para Dataset Hugging Face

```
# Converter DataFrames para Dataset Hugging Face
train_dataset = Dataset.from_pandas(train_df[["Translated_Review", "label"]])
test_dataset = Dataset.from_pandas(test_df[["Translated_Review", "label"]])
```

```
# Mostrar o conteúdo
display(train_dataset)
display(test_dataset)
```

```
Dataset({
    features: ['Translated_Review', 'label', '__index_level_0__'],
    num_rows: 25815
})
Dataset({
    features: ['Translated_Review', 'label', '__index_level_0__'],
    num_rows: 6454
})
```

3.4. Tokenização dos Conjuntos de Treino e Teste

```
# Carregar tokenizador DistilBERT
tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-uncased")

# Tokenizar treino
train_encodings = tokenizer(
    list(train_dataset["Translated_Review"]),
    truncation=True,
    padding=True,
    max_length=512
)

# Tokenizar teste
test_encodings = tokenizer(
    list(test_dataset["Translated_Review"]),
    truncation=True,
    padding=True,
    max_length=512
)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:112: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Co
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
```

```
# Adicionar tokens ao treino
train_dataset = train_dataset.add_column("input_ids", train_encodings["input_ids"])
train_dataset = train_dataset.add_column("attention_mask", train_encodings["attention_mask"])
```

```
# Adicionar tokens ao teste
test_dataset = test_dataset.add_column("input_ids", test_encodings["input_ids"])
test_dataset = test_dataset.add_column("attention_mask", test_encodings["attention_mask"])
```

3.5. Carregamento do Modelo e Definição dos Parâmetros de Treino

```
# Converter datasets para formato PyTorch
train_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])
test_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])

# Carregar modelo DistilBERT
model = DistilBertForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=2
)

# Definir parâmetros de treino
training_args = TrainingArguments(
    output_dir="./output",
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    num_train_epochs=2,
    report_to="none"
)
```

```
Loading weights: 100%                                                    100/100 [00:00<00:00, 482.58it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                     | Status     |
------------------------+------------+-
vocab_layer_norm.bias   | UNEXPECTED |
vocab_projector.bias    | UNEXPECTED |
vocab_transform.bias    | UNEXPECTED |
vocab_transform.weight  | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
pre_classifier.weight   | MISSING    |
classifier.weight       | MISSING    |
classifier.bias         | MISSING    |
pre_classifier.bias     | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
```

3.6. Definição da Métrica de Avaliação

```
def calcular_metricas(eval_pred):

    logits, labels = eval_pred

    preds = np.argmax(logits, axis=1)

    acc = accuracy_score(labels, preds)
    prec = precision_score(labels, preds, average='weighted')
    rec = recall_score(labels, preds, average='weighted')
    f1 = f1_score(labels, preds, average='weighted')
    kappa = cohen_kappa_score(labels, preds)

    cm = confusion_matrix(labels, preds)

    tn, fp, fn, tp = cm.ravel()

    taxa_erro = (fp + fn) / (tp + tn + fp + fn)

    return {
        "accuracy": acc,
        "precision": prec,
        "recall": rec,
        "f1": f1,
        "kappa": kappa,
        "taxa_erro": taxa_erro
    }
```

3.7. Treino e Avaliação do Modelo

```
# Configurar treino
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=calcular_metricas
)

# Treinar modelo
trainer.train()
```

[7940/12908 30:56 < 19:21, 4.28 it/s, Epoch 1.23/2]

Step	Training Loss
500	0.460684
1000	0.376153
1500	0.354738
2000	0.329143
2500	0.323698
3000	0.311494
3500	0.290749
4000	0.296565
4500	0.260710
5000	0.236775

```
# Avaliar modelo
resultados = trainer.evaluate()

# Converter resultados para DataFrame
df_resultados = pd.DataFrame([{
    "Model": "DistilBERT",
    "Acc": resultados["eval_accuracy"],
    "Prec": resultados["eval_precision"],
    "Rec": resultados["eval_recall"],
    "F1": resultados["eval_f1"],
    "Kappa": resultados["eval_kappa"],
    "Taxa_Erro": resultados["eval_taxa_erro"]
}])

# Mostrar tabela
print(df_resultados)

# Exportar CSV
df_resultados.to_csv("DistilBERT_results.csv", index=False)
```

Writing model shards: 100% 1/1 [00:01<00:00, 1.48s/it]

[1614/1614 00:57]

Writing model shards: 100% 1/1 [00:02<00:00, 2.79s/it]

	Model	Acc	Prec	Rec	F1	Kappa	Taxa_Erro
0	DistilBERT	0.964208	0.964663	0.964208	0.964371	0.906937	0.035792

Writing model shards: 100% 1/1 [00:10<00:00, 10.01s/it]

Writing model shards: 100% 1/1 [00:08<00:00, 8.06s/it]

Writing model shards: 100% 1/1 [00:09<00:00, 9.07s/it]

4. Predição de Novo Comentário

```
### --- NOVOS COMENTÁRIOS: --- ###

uploaded = files.upload()

novos_comentarios = pd.read_csv(io.BytesIO(uploaded['Comentarios.csv']), sep=';')
```

Escolher Ficheiros Comentarios.csv

Comentarios.csv(text/csv) - 255 bytes, last modified: 12/05/2026 - 100% done

```
Saving Comentarios.csv to Comentarios.csv
```

4.1. Pré-processamento

```
# Remover reviews com sentimento neutro
novos_comentarios = novos_comentarios[novos_comentarios['Sentiment'] != 'Neutral']

# Visualizar as primeiras linhas
display(novos_comentarios.head(10))
```

	Comentarios_Novos	Sentiment
0	This app is a true paragon.	Positive
1	The interface remains totally execrable.	Negative
3	Simply effulgent and highly efficient!	Positive
4	Navigation is inimical to usage.	Negative

4.2. Avaliação de Novos Comentários

```
# Extrair textos
textos = novos_comentarios["Comentarios_Novos"].astype(str).tolist()

# Tokenizar em batch
tokens = tokenizer(
    textos,
    truncation=True,
    padding=True,
    max_length=128,
    return_tensors="pt"
)

# Garantir que o modelo está na GPU (se existir)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

```
# Enviar tokens para o mesmo device
tokens = {k: v.to(device) for k, v in tokens.items()}

# Colocar modelo em modo avaliação
model.eval()

# Inferência
with torch.no_grad():
    outputs = model(**tokens)

logits = outputs.logits

# Previsões
classe_prevista = np.argmax(logits.cpu().numpy(), axis=1)

# Converter para labels originais
sentimento = le.inverse_transform(classe_prevista)

# Guardar no dataframe
novos_comentarios["Sentimento_previsto"] = sentimento

# Mostrar resultados
print(novos_comentarios[["Comentarios_Novos", "Sentiment", "Sentimento_previsto"]])
```

```
                        Comentarios_Novos Sentiment Sentimento_previsto
0              This app is a true paragon.  Positive            Positive
1  The interface remains totally execrable.  Negative            Negative
3    Simply effulgent and highly efficient!  Positive            Positive
4          Navigation is inimical to usage.  Negative            Negative
```

5. eXplainable AI

```
!pip install shap -q
```

```
# ============================================================
# EXPLAINABLE AI - SHAP PARA DISTILBERT (VERSÃO ULTRA-BLINDADA)
# ============================================================

# 1. Mover o modelo para o CPU (evita estouro de memória no Colab com o SHAP)
device = torch.device("cpu")
model.to(device)
model.eval()

# 2. Função de previsão robusta que força a conversão para lista de strings
def predict_probabilities(texts):

    if isinstance(texts, np.ndarray):
        texts = texts.tolist()
    if isinstance(texts, str):
        texts = [texts]

    cleaned_texts = [str(t) for t in texts]

    inputs = tokenizer(
        cleaned_texts,
        padding=True,
        truncation=True,
        max_length=128,
        return_tensors="pt"
    ).to(device)

    with torch.no_grad():
        outputs = model(**inputs)

    probabilities = torch.nn.functional.softmax(outputs.logits, dim=-1)
    return probabilities.cpu().numpy()

# 3. Inicializar o Explainer específico para Texto com a função robusta
explainer = shap.Explainer(predict_probabilities, masker=shap.maskers.Text(tokenizer))

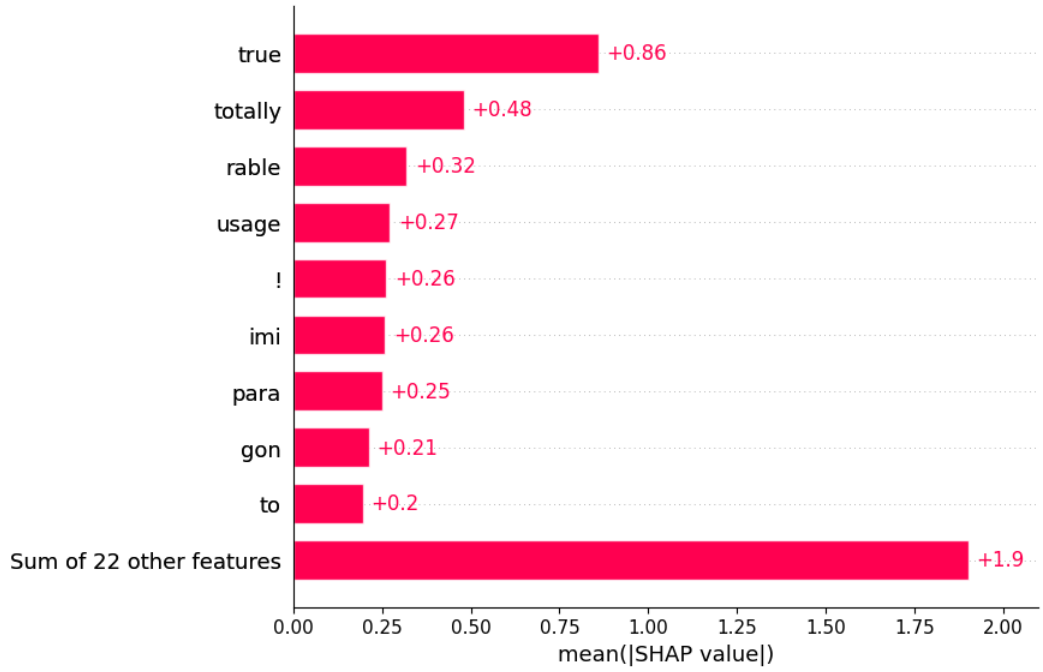
# 4. Preparar estritamente a amostra de strings
text_samples = [str(t) for t in textos if pd.notna(t)]
text_samples = [t for t in text_samples if t.strip() != ""]

# Selecionamos apenas as primeiras 15 para testar rápido (SHAP em LLMs demora)
text_samples = text_samples[:15]

print("A calcular SHAP values globais para o DistilBERT...")
shap_values = explainer(text_samples)

# 5. Gerar o gráfico global de barras
print("A gerar gráfico global SHAP...")
shap.plots.bar(shap_values[:, :, 1])
```

```
A calcular SHAP values globais para o DistilBERT...
PartitionExplainer explainer:  75%|███████▌  | 3/4 [00:16<00:02,  2.33s/it]
PartitionExplainer explainer: 100%|██████████| 4/4 [00:22<00:00,  3.79s/it]
PartitionExplainer explainer: 5it [00:27,  6.89s/it]
A gerar gráfico global SHAP...
```




```python
# ============================================================
# TOP FEATURES MAIS IMPORTANTES - DISTILBERT (CORREÇÃO DE ÍNDICE)
# ============================================================

all_tokens_data = []

# 1. Percorrer cada frase explicada para extrair os dados individualmente
for idx in range(len(shap_values)):

    frase_tokens = shap_values.feature_names[idx]

    valores_frase_completa = shap_values.values[idx]

    if valores_frase_completa.ndim == 2:
        frase_valores = valores_frase_completa[:, 1]
    else:
        frase_valores = valores_frase_completa

    # Guardar os pares (token, valor absoluto)
    for token, val in zip(frase_tokens, frase_valores):
        token_clean = str(token).strip()

        if token_clean not in ['', '[PAD]', '[CLS]', '[SEP]', '##', 'None']:
            all_tokens_data.append({
                'feature': token_clean,
                'importance': np.abs(val)
            })

# 2. Criar o DataFrame com todos os tokens agregados
df_raw_importance = pd.DataFrame(all_tokens_data)

# 3. Agrupar por palavra/token e calcular a média real
importance_df = df_raw_importance.groupby('feature', as_index=False).mean()

# 4. Ordenar do maior impacto para o menor
importance_df = importance_df.sort_values(
    by='importance',
    ascending=False
)

print("\nTop 20 tokens/palavras mais importantes para o DistilBERT:\n")
print(importance_df.head(20).to_string(index=False))
```

```
Top 20 tokens/palavras mais importantes para o DistilBERT:

  feature  importance
     true    0.860624
  totally    0.479737
    rable    0.319677
    usage    0.272349
        !    0.261973
      imi    0.258589
     para    0.250290
      gon    0.212532
       to    0.197208
       ex    0.187751
        e    0.169227
efficient    0.164921
   highly    0.161872
       in    0.153893
      cal    0.150932
       ul    0.143785
```

```
       ff    0.138428
interface    0.096050
        a    0.085210
       is    0.081920
```

```
# ============================================================
#  GUARDAR RESULTADOS - DISTILBERT
# ============================================================

importance_df.to_csv(
    "distilbert_shap_feature_importance.csv",
    index=False
)

print("Ficheiro guardado:")
print("distilbert_shap_feature_importance.csv")
```

```
# ============================================================
# Comentário positivo e negativo do df_final e comentário extraído do CSV
# ============================================================

texto_positivo = "great finding place take right door step place want go worked great warsaw aire"
texto_negativo = "track food linked since fitness food pound may seem small big"
texto_novo     = "interface remains totally execrable"

todas_as_frases = [texto_positivo, texto_negativo, texto_novo]

# ============================================================
# O LOOP QUE ANALISA AS 3 FRASES AUTOMATICAMENTE - DISTILBERT
# ============================================================

for frase_escolhida in todas_as_frases:
    print("\n" + "="*70)
    print(f" A ANALISAR: \"{frase_escolhida}\"")
    print("="*70)

    # ============================================================
    # CALCULAR SHAP VALUES
    # ============================================================
    shap_explanation = explainer([frase_escolhida])

    shap_values_for_plot = shap_explanation[0, :, 1]

    # ============================================================
    # GERAR EXPLICAÇÃO VISUAL
    # ============================================================
    print("\nA gerar explicação SHAP...\n")

    shap.plots.waterfall(shap_values_for_plot)

    # ============================================================
    # MOSTRAR PALAVRAS MAIS IMPORTANTES
    # ============================================================
    feature_names = shap_values_for_plot.feature_names
    valores = shap_values_for_plot.values

    indices = np.argsort(np.abs(valores))[::-1]

    print("\nTOP TOKENS MAIS IMPORTANTES:\n")

    for i in indices[:10]:
        if valores[i] != 0:
            palavra = str(feature_names[i]).strip()
            impacto = valores[i]

            # Evitar poluir o terminal com tokens vazios ou de preenchimento
            if palavra not in ['', '[PAD]', '[CLS]', '[SEP]', '##', 'None']:
                print(f"'{palavra}' ---> impacto SHAP = {impacto:.4f}")

    # ============================================================
    # EXPLICAÇÃO TEXTUAL AUTOMÁTICA
    # ============================================================
    print("\nEXPLICAÇÃO AUTOMÁTICA:\n")

    for i in indices[:10]:
        if valores[i] != 0:
            palavra = str(feature_names[i]).strip()
            impacto = valores[i]

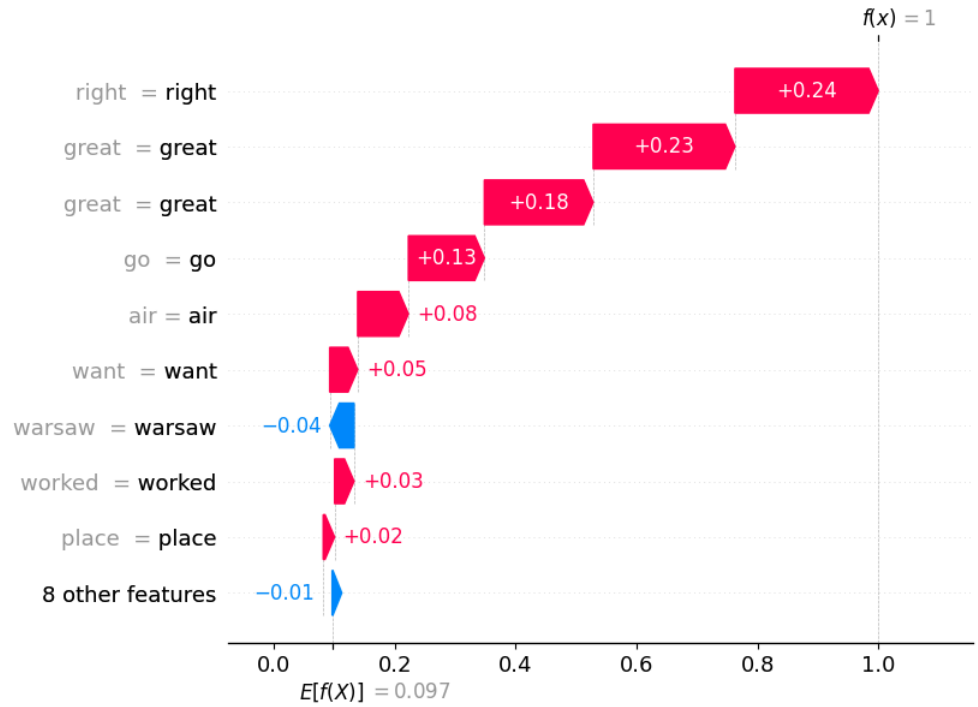
            if palavra not in ['', '[PAD]', '[CLS]', '[SEP]', '##', 'None']:
                if impacto > 0:
                    direcao = "POSITIVA"
                else:
                    direcao = "NEGATIVA"

                print(
                    f"O token '{palavra}' "
                    f"teve influência {direcao} "
                    f"na classificação."
                )
```

```
======================================================================
 A ANALISAR: "great finding place take right door step place want go worked great warsaw aire"
======================================================================
```

76% 208/272 [00:03<00:00, 646.46it/s]

A gerar explicação SHAP...



TOP TOKENS MAIS IMPORTANTES:

```
'right' ---> impacto SHAP = 0.2372
'great' ---> impacto SHAP = 0.2345
'great' ---> impacto SHAP = 0.1800
'go' ---> impacto SHAP = 0.1256
'air' ---> impacto SHAP = 0.0839
'want' ---> impacto SHAP = 0.0463
'warsaw' ---> impacto SHAP = -0.0403
'worked' ---> impacto SHAP = 0.0322
'place' ---> impacto SHAP = 0.0188
'e' ---> impacto SHAP = 0.0052
```

EXPLICAÇÃO AUTOMÁTICA:

```
O token 'right' teve influência POSITIVA na classificação.
O token 'great' teve influência POSITIVA na classificação.
O token 'great' teve influência POSITIVA na classificação.
O token 'go' teve influência POSITIVA na classificação.
O token 'air' teve influência POSITIVA na classificação.
O token 'want' teve influência POSITIVA na classificação.
O token 'warsaw' teve influência NEGATIVA na classificação.
O token 'worked' teve influência POSITIVA na classificação.
```